Smart Personalized Product Recommendation System

Table of Contents

01 System Objective & Overview 3

02 High-Level Architecture 5

03 Development Workflow 6

04 Technology Stack 7

05 Core System Modules 8

06 User Interaction Flow 10

07 API Design & Integration 11

08 Testing & Deployment 12

09 Performance & Security 13

10 Project Deliverables 14

1 System Objective & Overview

Executive Summary

Building a **multi-channel conversational recommendation system** that functions as a digital AI-assisted sales assistant, engaging users in real-time across WhatsApp, Instagram, and Web platforms.

Primary Objectives

🎯

PREFERENCE COLLECTION

100%

Data capture rate

⚡

PROCESSING SPEED

< 2s

Response time

📱

CHANNEL COVERAGE

3

Platforms

End-to-End User Flow

1. User opens platform (WhatsApp / Instagram / Web)

↓

2. System starts guided questionnaire (5-8 questions)

↓

3. Instant processing & intelligent matching

↓

4. **3-Tier Recommendations Delivered** (Basic → Intermediate → Premium)

Core Experience Principles

PRINCIPLE	TARGET	MEASUREMENT
⚡ **Fast**	~2 minutes completion	Session timing logs
✨ **Simple**	Conversational flow	User drop-off rate
🎯 **Personalized**	Based on user input	Response accuracy
✅ **Actionable**	Clear product suggestions	Click-through rate

2 High-Level Architecture

System Components

📱 Channel Layer

WhatsApp (Twilio) | Instagram (Meta API) | Web Interface

↓

⚙️ Backend Server (Node.js)

Questionnaire Engine | Session Management | Recommendation Engine | Offer Generation

↓

🗄️ Data Layer

Product Catalog | Session Store | Analytics Logs

Data Flow Architecture

STEP	ACTION	COMPONENT
1	User message received	Chat API
2	Request processing	Backend Server
3	Questionnaire flow	Questionnaire Engine
4	Data persistence	Session Store
5	Product matching	Recommendation Engine

STEP	ACTION	COMPONENT
6	Response formatting	Offer Generator

3 Development Workflow

6-Week Implementation Plan

PHASE 1 FOUNDATION

Project Setup

Initialize backend, define product schema, design questionnaire flow

PHASE 2 CORE

Core Logic

Build questionnaire engine, session handling, recommendation algorithm

PHASE 3 INTEGRATION

Chat Integration

WhatsApp webhook, Instagram webhook, connect to core logic

PHASE 4 LOGIC

Offer System

3-tier generation logic, response formatting, testing

PHASE 5 FRONTEND

Web Interface

Frontend development, API integration, UI/UX polish

PHASE 6 DEPLOY

Testing & Deploy

E2E testing, bug fixes, production deployment

Key Activities by Phase

PHASE	DURATION	KEY DELIVERABLES
Setup & Planning	Week 1	Project structure, Data models, Question bank
Core Logic	Week 2	Working engine, Session manager, Matching algorithm
Integration	Week 3	WhatsApp & Instagram webhooks functional
Offer System	Week 4	3-tier recommendation system complete
Web Interface	Week 5	Functional web application

PHASE	DURATION	KEY DELIVERABLES
Testing & Deploy	Week 6	Live production system

4 Technology Stack

LAYER	TECHNOLOGY	JUSTIFICATION
Backend	Node.js (Express/Fastify)	Fast, scalable, JavaScript ecosystem
Database	JSON / PostgreSQL	Flexible start, enterprise scalable
WhatsApp	Twilio API	Reliable, well-documented
Instagram	Meta Messenger API v18.0	Official Meta integration
Hosting	Railway / AWS / VPS	Auto-scaling, easy deployment
Monitoring	LogRocket / Sentry	Error tracking, session replay

Technical Specifications

20.x LTS
NODE.JS VERSION

16.x
POSTGRESQL VERSION

REST
API ARCHITECTURE

JWT
AUTHENTICATION

5 Core System Modules

5.1 Questionnaire Engine

Handles user interaction and collects structured data through multiple input types.

```
// Supported Question Types { "types": [ "multiple_choice", // Budget, Category selection "text_input", // Open-ended features "conditional" // Logic-based branching ] }
```

5.2 Session Management

Tracks user progress and maintains conversation state across all channels.

- Current question index
- User responses array
- Completion state
- Channel identification

5.3 Recommendation Engine

Phase 1: Rule-Based Matching

Tag matching → Scoring system → Tiered grouping

5.4 Offer Generation Tiers

TIER	FORMAT	EXAMPLE
Basic	1-2 simple options	"Try: Basic Headphones ($49) or Sport Earbuds ($79)"
Intermediate	3-4 options + reasoning	"Premium Sound ($199) for audiophiles, Compact Pro ($149) for portability"
Premium	Best match + bundle	"Ultimate Bundle: Headphones + Amp + Warranty at 20% off"

5.5 Chat Handler Logic

```
// Conversation State Machine IF new_user → start_questionnaire() IF
incomplete → ask_next_question() IF completed →
generate_recommendations()
```

6 User Interaction Flow

Detailed Conversation Flow

1. **Entry:** User sends message → Session initialized

↓

2. **Questionnaire Loop:** System asks → User responds → Data stored

↓

3. **Processing:** Answers converted to tags → Matched with products

↓

4. **Output:** 3-tier recommendations generated and sent

Questionnaire Structure

STEP	QUESTION FOCUS	INPUT TYPE
1	Budget Range	Multiple Choice
2	Product Category	Multiple Choice
3	Key Features	Text / Select
4	Brand Preference	Multiple Choice
5	Urgency / Timeline	Multiple Choice
6-8	Conditional Follow-ups	Dynamic

7 API Design & Integration

Core API Endpoints

POST	`/session/start`	Initialize new user session
POST	`/session/answer`	Submit questionnaire answer
GET	`/session/next`	Retrieve next question
GET	`/recommendations`	Get 3-tier personalized offers

Multilingual Support

🇵🇹 **PT**
PRIMARY LANGUAGE

🇬🇧 **EN**
SECONDARY READY

Integration Architecture

📱 **Messaging Platforms**

Twilio Webhook | Meta Messenger API | REST API

↓

🔐 **API Gateway**

Authentication | Rate Limiting | Request Validation

8 Testing & Deployment

Testing Strategy

TYPE	FOCUS AREAS	TOOLS
Functional	Questionnaire flow, Recommendation accuracy, Offer generation	Jest, Mocha
Integration	WhatsApp API, Instagram API, Database connectivity	Postman, Newman
E2E	Full conversation flows, Edge cases, Error handling	Cypress, Playwright

Edge Case Handling

- ✓ Invalid input validation and user re-prompting
- ✓ Session timeout management (30-minute expiration)
- ✓ Missing or incomplete data recovery
- ✓ API failure fallback mechanisms
- ✓ Concurrent user session isolation

Deployment Steps

1. Deploy backend server to cloud infrastructure
2. Configure environment variables and secrets
3. Connect messaging APIs (Twilio, Meta)
4. Test webhook endpoints and verify signatures

5 **Launch production system**

9 Performance & Security

Performance Targets

⚡

RESPONSE TIME

< 2s

95th percentile

👥

CONCURRENT USERS

1000+

Per instance

📈

UPTIME SLA

99.9%

Availability

Security Implementation

- ✓ Input validation on all endpoints with schema validation (Joi/Zod)
- ✓ Rate limiting: 100 requests/minute per user
- ✓ API key authentication with JWT tokens
- ✓ WhatsApp webhook signature verification (Twilio)
- ✓ Instagram Messenger signature validation (Meta)
- ✓ Environment-based secrets management (dotenv)
- ✓ Session timeout: 30 minutes automatic expiration
- ✓ CORS configuration for allowed origins only

Optimization Strategies

STRATEGY	IMPLEMENTATION	IMPACT
Caching	Product data in-memory cache	Reduce DB queries by 80%

STRATEGY	IMPLEMENTATION	IMPACT
Connection Pooling	Database connection reuse	Faster query response
Lazy Loading	Load recommendations on-demand	Reduced memory footprint

10 Project Deliverables

Final Delivery Package

- ✓ **Backend System** — Complete Node.js application with modular architecture
- ✓ **Chat Integrations** — Production-ready WhatsApp (Twilio) and Instagram (Meta) connectors
- ✓ **Recommendation Engine** — Rule-based matching algorithm with scoring system
- ✓ **Questionnaire System** — Dynamic flow engine with conditional logic support
- ✓ **Offer Generation Module** — 3-tier recommendation logic with customizable templates
- ✓ **Web Interface** — Responsive frontend application (React/Vanilla JS)
- ✓ **API Documentation** — Complete endpoint documentation with examples
- ✓ **Deployment Guide** — Step-by-step production deployment instructions
- ✓ **Monitoring Setup** — Error tracking (Sentry) and analytics (LogRocket) configuration

Documentation Coverage

This document provides **complete clarity for implementation without ambiguity**, covering system architecture, development workflow, user interaction flow, backend logic, integration structure, and deployment process.